

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica For The Win

14 de octubre de 2024

Alejo Sendra
107716

Agustín Murseli
107752

Leticia Aab
106053

1. Introducción

En el presente trabajo nos enfocaremos en el desarrollo y análisis de un algoritmo de programación dinámica que ayude a Sophia a maximizar su ganancia en un juego de selección de monedas y, siempre que sea posible, vencer a su hermano, Mateo.

2. Analisis del problema

2.1. Consideraciones

Para la resolución del problema planteado consideramos importante resaltar los siguientes puntos:

- El problema puede abstraerse a un arreglo con los valores de las monedas, del cual iremos removiendo el primer o último elemento.
- Los valores se remueven de a uno por turno, el primero o el último, y gana quien obtenga mayor ganancia, por sumatoria.
- Al tratarse de monedas, los valores siempre serán superiores a 0.
- Sophia siempre comienza primero.
- Mateo usará una estrategia greedy: en su turno siempre elegirá la moneda de mayor valor entre la primera y la última.
- Existirán casos en los que Sophia no podrá ganar, no obstante, el objetivo es maximizar su ganancia siempre.

2.2. Casos base

Para ayudarnos en la resolución de un caso representativo del problema, primero analicemos las soluciones, para Sophia, de los casos base, o triviales, del problema:

- En caso de tener 0 monedas la única solución posible sería 0, no hay monedas a elegir.
- En caso de tener 1 moneda la única solución posible es el valor de la moneda.
- Cuando tenemos 2 monedas, aprovechando que Sophia siempre tiene el primer turno, se quedará la de mayor valor.

2.3. Exploración de las solución

Teniendo en cuenta lo anterior, planteemos algunas soluciones óptimas para Sophia, pero esta vez hagamos los seguimientos, considerando "m" nuestro arreglo de monedas:

Caso 1:

$$m = [4, 10, 2]$$

1er turno

$$m = [10, 2]$$

$$\text{Sol. Sophia} = 4$$

$$\text{Sol. Mateo} =$$

2do turno

$$m = [2]$$

$$\text{Sol. Sophia} = 4$$

$$\text{Sol. Mateo} = 10$$

3er turno

$$m = []$$

$$\text{Sol. Sophia} = 4 + 2$$

$$\text{Sol. Mateo} = 10$$

Notar que sin importar si Sophia agarra la moneda de la izquierda o de la derecha, siempre ganará Mateo con una ganancia de 10 y ella perderá con una ganancia máxima de 6. De este caso particular se puede observar que el algoritmo no puede asegurar la victoria de Sophia y se concluye que para toda situación donde se tengan tres monedas Mateo ganará si y solo si, dado un arreglo de monedas "m":

$$m = [m_0, m_1, m_2]$$

Se cumple que:

$$m_1 > m_0 + m_2$$

Caso 2:

$$m = [5, 15, 3, 7]$$

1er turno

$$m = [5, 15, 3]$$

$$\text{Sol. Sophia} = 7$$

$$\text{Sol. Mateo} =$$

2do turno

$$m = [15, 3]$$

$$\text{Sol. Sophia} = 7$$

$$\text{Sol. Mateo} = 5$$

3er turno

$$m = [3]$$

$$\text{Sol. Sophia} = 7 + 15$$

$$\text{Sol. Mateo} = 5$$

4to turno

$$m = []$$

$$\text{Sol. Sophia} = 7 + 15 = 22$$

$$\text{Sol. Mateo} = 5 + 3 = 8$$

De este caso podríamos pensar que la mejor estrategia que podría tener Sophia es imitar a Mateo y elegir la moneda de mayor valor. Pero esta solución no garantiza que Sophia obtenga la máxima suma posible, veamos otro caso

Caso 3:

$$m = [4, 10, 5, 2]$$

1er turno

$$m = [4, 10, 5]$$

$$\text{Sol. Sophia} = 2$$

$$\text{Sol. Mateo} =$$

2do turno

$$m = [4, 10]$$

$$\text{Sol. Sophia} = 2$$

$$\text{Sol. Mateo} = 5$$

3er turno

$$m = [4]$$

$$\text{Sol. Sophia} = 2 + 10$$

$$\text{Sol. Mateo} = 5$$

4to turno

$$m = []$$

$$\text{Sol. Sophia} = 2 + 10$$

$$\text{Sol. Mateo} = 5 + 4$$

Notar que si Sophia elegía la moneda de mayor valor en la primera ronda, hubiera perdido. Logró ganar ya que tomó en cuenta las posibles jugadas de Mateo, si ella elegía la moneda de valor 4, habilitaba a Mateo a obtener la moneda de valor 10. Es claro que a Sophia le convenía elegir la moneda de valor 2, Mateo luego elegiría la de valor 5, y la moneda 10 se la quedaría ella, logrando maximizar la suma de los valores de sus monedas.

2.4. Ecuación de Recurrencia

Mirando el desarrollo anterior y los casos base, podemos comenzar a armar nuestra ecuación de recurrencia.

Sea el conjunto de monedas $m_0, m_1, \dots, m_n$.

Definimos a la moneda de la izquierda en cada turno como m_i a la moneda de la derecha en cada turno como m_d .

Podemos definir un problema $OPT(i, d)$ que represente la ganancia máxima que puede obtener Sophia cuando las monedas disponibles son m_i hasta m_d .

En su turno, Sophia tiene dos opciones, tomar la moneda en la posición i o la moneda en la posición d , debiendo elegir aquella que le asegure una ganancia maxima a futuro, cuando sea su turno nuevamente y el arreglo de monedas sea de longitud $n-2$, con lo cual podriamos decir que la ecuación de recurrencia tendría la siguiente estructura:

$$OPT(i, d) = \max(m_i + OPT[n-2], m_d + OPT[n-2])$$

Si Sophia toma la moneda de la izquierda, Mateo en su turno se quedará con el máximo entre m_{i+1} y m_d , y luego Sophia enfrentará el subproblema de tamaño más pequeño que puede ser la secuencia $m_{i+2} \dots m_d$ o $m_{i+1} \dots m_{d-1}$ dependiendo de la decisión previa de su hermano.

Si Sophia toma m_d , Mateo elegirá la moneda de mayor valor entre m_i y m_{d-1} , y el juego continuará análogamente al caso anterior. Concluimos entonces que una posible ecuacion de recurrencia sería:

- Para una moneda: $OPT(i, i) = m_i$

■ Para dos monedas: $OPT(i, i+1) = \text{máx}(m_i, m_{i+1})$

■ Para n monedas:

$$OPT(i, d) = \text{máx} \left(m_i + \begin{cases} OPT(i+2, d) & \text{si } m_{i+1} > m_d \\ OPT(i+1, d-1) & \text{si } m_d > m_{i+1} \end{cases}, m_d + \begin{cases} OPT(i+1, d-1) & \text{si } m_i > m_{d-1} \\ OPT(i, d-2) & \text{si } m_{d-1} > m_i \end{cases} \right)$$

3. Resolución

A continuación realizaremos el análisis teórico y empírico del algoritmo propuesto para resolver el juego de las monedas por programación dinámica.

3.1. Algoritmo Iterativo

A continuación se muestra el código de solución iterativa del problema.

```
1 def coins_game(coins):
2     n = len(coins)
3     if n == 0:
4         return 0,0,[]
5
6     dp = [[0] * n for _ in range(n)]
7
8     for i in range(n):
9         dp[i][i] = coins[i] #Caso base: una sola moneda
10        if i < n - 1:
11            dp[i][i + 1] = max(coins[i], coins[i + 1]) #Caso base: dos monedas
12
13    for d in range(2, n):
14        for i in range(n - d):
15            j = i + d
16            if coins[i+1] > coins[j]:
17                dp[i][j] = coins[i] + dp[i+2][j]
18            else:
19                dp[i][j] = coins[i] + dp[i+1][j-1]
20
21            if coins[j-1] > coins[i]:
22                dp[i][j] = max(dp[i][j], coins[j] + dp[i][j-2])
23            else:
24                dp[i][j] = max(dp[i][j], coins[j] + dp[i+1][j-1])
25
26    sophia_gain = dp[0][n-1]
27    mateo_gain = sum(coins) - sophia_gain
28
29    return sophia_gain, mateo_gain, dp
```

Para la solución proponemos la creación de una matriz. En primer lugar insertamos en la diagonal los valores que corresponden al índice en el arreglo original (ej. en la posición 1,1 de la matriz iría el elemento 1 del arreglo), dado que la mejor opción para Sophia es simplemente tomar esa moneda si consideramos esa moneda como la única disponible, así mismo, insertamos a derecha de la diagonal el maximo entre cada valor y su siguiente. finalmente vamos completando los espacios restantes de la matriz según el óptimo de los sub-arreglos de largo creciente hasta completar el óptimo del arreglo de largo n cuyo resultado se encontrara en la posicion $[0][n]$ de nuestra matriz.

El algoritmo es de programación dinámica por que resuelve un problema grande (maximizar la ganancia de Sophia para n monedas) combinando las soluciones de subproblemas más pequeños previamente resueltos y memorizados (maximizar la ganancia de Sophia cuando hay menos monedas).

3.2. Reconstrucción de la solución

A continuación presentamos el algoritmo que reconstruye la solución a partir de la matriz obtenida anteriormente y el arreglo de monedas original.

```
1 def reconstruct_solution(coins, dp):
2     n = len(coins)
3     i, j = 0, n - 1
4     is_sophia_turn = True
5
6     while i <= j:
7         if is_sophia_turn:
8             first = coins[i] + min(dp[i+2][j] if i+2 <= j else 0, dp[i+1][j-1] if i+1 <= j-1
9             else 0)
10            if first >= dp[i][j]:
11                print(f"Sophia debe agarrar la primera ({coins[i]}); ", end="")
12                i += 1
13            else:
14                print(f"Sophia debe agarrar la ultima ({coins[j]}); ", end="")
15                j -= 1
16        else:
17            if coins[i] > coins[j]:
18                print(f"Mateo agarra la primera ({coins[i]}); ", end="")
19                i += 1
20            else:
21                print(f"Mateo agarra la ultima ({coins[j]}); ", end="")
22                j -= 1
23        is_sophia_turn = not is_sophia_turn
```

Cuando es el turno de Sophia, se calculan dos posibles escenarios: tomar la primera moneda o la última. Para decidir, compara los valores que obtiene en ambos casos, utilizando los resultados en la tabla dp. La línea 8, que contiene `min()`, es clave, ya que representa la mejor jugada que puede hacer Mateo en su turno, dado que él también tomará monedas. Si la mejor opción es tomar la primera moneda, se avanza el índice *i*, de lo contrario, se toma la última y se decrementa *j*. Mateo, en su turno, simplemente elige la moneda de mayor valor entre los extremos y el juego continúa alternando hasta que se hayan tomado todas las monedas (es decir, hasta que el índice *i* sea mayor que *j*).

3.3. Complejidad

3.3.1. Complejidad del algoritmo solución

Con la explicación anteriormente dada, y el código presentado, desarrollemos la complejidad del algoritmo planteado analizando las partes más importantes de nuestra solución considerando " n " como la cantidad de monedas:

- Creación de una matriz de $n \times n$: $O(n^2)$

```
1 dp = [[0] * n for _ in range ( n ) ]
```

- Inicialización de los valores de la diagonal de la matriz y de sus valores continuos $O(n)$:

```
1 for i in range(n):
2     dp[i][i] = coins[i] #Caso base: una sola moneda
3     if i < n - 1:
4         dp[i][i + 1] = max(coins[i], coins[i + 1]) #Caso base: dos m
```

- Ahora analicemos los loops principales del algoritmo, desestimando las comparaciones que internamente se realizan en dichos loops y cuya complejidad es $O(1)$.

```
1 for d in range (2 , n ) :
2     for i in range ( n - d ) :
```

Podemos observar que el primer loop se repetirá " $n - 2$ " veces mientras que el segundo se repetirá una cantidad decreciente de veces, ya que " d " va incrementando. Entonces la complejidad temporal introducida por dicha porción de código, sería algo como:

$$T(n) = (n - 2) + (n - 3) + (n - 4) + \dots + 1$$

Lo cual podríamos reescribir como la sumatoria:

$$\begin{aligned} \sum_{i=2}^{n-1} (n - i) &= \sum_{i=2}^{n-1} n - \sum_{i=2}^{n-1} i = \sum_{i=1}^{n-2} n - \sum_{i=1}^{n-2} i \\ &= n(n - 2) - (2 + (n - 1)) \frac{(n - 2)}{2} \\ &= n^2 - 2n - \frac{n^2}{2} + \frac{n}{2} + 1 = \frac{n^2}{2} - \frac{3n}{2} + 1 \end{aligned}$$

Lo cual podemos acotar a:

$$O(n^2)$$

De esta forma la complejidad del algoritmo solución resulta ser:

$$\begin{aligned} T(n) &= O(n^2) + O(n) + O(n) + O(n^2) \\ T(n) &= O(n^2) \end{aligned}$$

3.3.2. Complejidad de la reconstrucción de la solución

Si analizamos el algoritmo que reconstruye la solución del problema vemos que presenta una complejidad lineal. El bucle *while* se ejecuta n veces, ya que en cada iteración se toma una moneda y se reduce el rango entre i y j . Las operaciones dentro del bucle, como el acceso a la tabla dp , los cálculos de mínimo, las comparaciones y los prints, son todas operaciones constantes $O(1)$. Por ende, la complejidad de esta función es $O(n)$.

4. Demostración de optimalidad

Para demostrar que el algoritmo planteado es óptimo, demostraremos por inducción que la ecuación de recurrencia en la cual se basa es óptima. Sea la secuencia de monedas $m_i, m_{i+1}, \dots, m_{d-1}, m_d$, todas diferentes y mayores a cero. Definimos $OPT(i, d)$ como la ganancia máxima que puede obtener Sophia con esas monedas:

Si hay solo una moneda : $OPT(i, i) = m_i$

Si hay dos monedas : $OPT(i, i+1) = \max(m_i, m_{i+1})$

Si hay "n" monedas :

$$OPT(i, d) = \max \left(m_i + \begin{cases} OPT(i+2, d) & \text{si } m_{i+1} > m_d \\ OPT(i+1, d-1) & \text{si } m_d > m_{i+1} \end{cases}, m_d + \begin{cases} OPT(i+1, d-1) & \text{si } m_i > m_{d-1} \\ OPT(i, d-2) & \text{si } m_{d-1} > m_i \end{cases} \right)$$

4.1. Hipótesis:

Para toda secuencia de monedas de longitud menor o igual a k , la ecuación de recurrencia encuentra la solución óptima.

4.2. Tesis:

La ecuación de recurrencia encuentra la solución óptima para una secuencia de monedas de longitud $k+1$.

4.2.1. Casos triviales

Cuando hay una sola moneda, Sophia la toma y maximiza su ganancia.

Si $n = k = d - i = 1 \Rightarrow OPT(i, i) = m_i$

Cuando hay dos monedas, Sophia toma la moneda de mayor, maximizando su ganancia

Si $n = k = d - i = 2 \Rightarrow OPT(i, d) = \max(m_i, m_d)$

4.2.2. Paso inductivo

$$Si \quad n = k + 1$$

Tenemos la secuencia de $k+1$ monedas: $m_i, m_{i+1}, \dots, m_{d-1}, m_d, m_{d+1}$, reemplazando en la ecuación, nos queda:

$$OPT(i, d+1) = \max \left(m_i + \begin{cases} OPT(i+2, d+1) & \text{si } m_{i+1} > m_{d+1} \\ OPT(i+1, d) & \text{si } m_{d+1} > m_{i+1} \end{cases}, m_{d+1} + \begin{cases} OPT(i+1, d) & \text{si } m_i > m_d \\ OPT(i, d-1) & \text{si } m_d > m_i \end{cases} \right)$$

Desarrollando los subproblemas, tenemos:

- $OPT(i+2, d+1)$ Sophia y Mateo eligieron la moneda de la izquierda, entonces, nos queda la secuencia de $k-1$ monedas: $m_{i+2}, \dots, m_{d-1}, m_d, m_{d+1}$
- $OPT(i+1, d)$ Sophia eligió la moneda de la izquierda y Mateo la de la derecha, o viceversa, entonces, nos queda la secuencia de $k-1$ monedas: $m_{i+1}, \dots, m_{d-1}, m_d$
- $OPT(i, d-1)$ Sophia y Mateo eligieron la moneda de la derecha, entonces, nos queda la secuencia de $k-1$ monedas: $m_i, m_{i+1}, \dots, m_{d-1}$.

En los tres subproblemas la longitud de la secuencia de monedas es menor a k , entonces por hipótesis podemos asegurar que encuentran la solución óptima para todos los posibles subproblemas resultantes.

$$\text{OPT}(i, d + 1) = \max(m_i + \text{Solución óptima del subproblema}, m_{d+1} + \text{Solución óptima del subproblema})$$

Considerando que nuestra solución conserva el máximo entre los dos caminos posibles en cada turno de Sophi (quedarse la primer moneda mas la solucion optima para ese caso o quedarse con la última moneda mas la solucion optima para ese caso) y que efectivamente reduce el problema en subproblemas más pequeños, entonces, podemos afirmar que encuentra la solución óptima del problema original.

Concluimos entonces que el algoritmo planteado permite maximizar la ganancia de Sophia siempre.

4.3. Análisis de variabilidad de valores

Notar que independientemente de la variabilidad de los valores de las monedas, el algoritmo ya plantea el camino más eficiente para lograr su objetivo teniendo en cuenta las dos posibilidades existentes en cada turno. Los valores absolutos tampoco afectan los tiempos de ejecución del algoritmo dado que las operaciones son constantes y solo depende del valor absoluto a muy bajo nivel.

5. Mediciones

Se realizaron mediciones en base a crear arreglos de diferentes tamaños, elegidos de forma uniforme entre 100 y 5.000 de elementos (utilizando la librería `numpy`). Donde los elementos en cada caso fueron generados por los valores pseudoaleatorios del lenguaje (el módulo `random`).

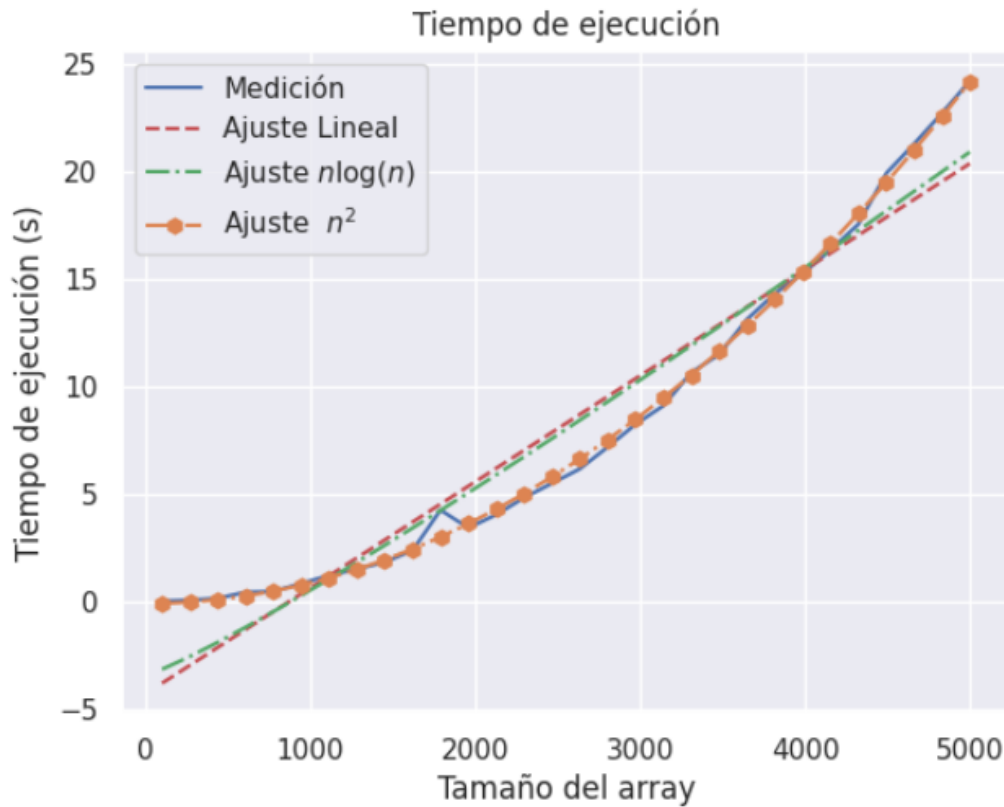


Figura 1. Comparación gráfica de la aproximación de la variabilidad temporal según el tamaño de entrada

De la figura 1 se puede observar efectivamente que el gráfico que nos aproxima a la variabilidad de los tiempos, de acuerdo al tamaño de la entrada, es una función cuadrática.



Figura 2. Comparación de los valores del error cuadrático medio al aproximar por diferentes funciones

Así mismo, en la figura 2 se puede apreciar el error absoluto producto de aproximar la variabilidad del algoritmo con diferentes funciones al utilizar el método de mínimos cuadrados.

5.1. ¿Cómo ejecutarlo?

Para correr el programa se puede usar el siguiente comando en una terminal:

```
python3 main.py <path coins value's file>
```

Donde <path coins value's file> es el path a un archivo .txt con la lista de monedas separadas por ",".

El detalle los ploteos y calculos utilizados para las mediciones se pueden encontrar en el Colab. El mismo se puede ejecutar en Entorno de ejecución ->Ejecutar todas.

Link al Colab: https://colab.research.google.com/drive/12ZQeVVanXeHReOh8QxjQ-u0FXVvRs_d1scrollTo=kbywChJiR_N7

6. Conclusiones

En este trabajo práctico, abordamos el problema de la selección de monedas entre dos jugadores, donde uno de ellos, Sophia, busca maximizar su ganancia mediante un enfoque óptimo; mientras que el otro, Mateo, sigue una estrategia simple basada en la selección de la moneda de mayor valor en cada turno. Utilizamos un algoritmo de Programación Dinámica para resolver el problema, aprovechando la descomposición del problema original en subproblemas más pequeños y almacenando sus soluciones intermedias en una tabla DP. Luego reconstruimos el problema utilizando un algoritmo que reproduce el juego simulando los turnos mientras mira la tabla.

El análisis de la complejidad temporal del algoritmo, tanto teórico como empírico, resultó ser $O(n^2)$, ya que para cada par de monedas, se calculan las soluciones utilizando los subproblemas previos. Esta complejidad es eficiente en comparación con una solución por fuerza bruta que exploraría todas las posibles secuencias de elección, lo que resultaría en un crecimiento exponencial.

A través de pruebas empíricas, verificamos que Sophia, al seguir este enfoque, puede asegurar una ganancia máxima, mientras que Mateo, al usar su estrategia greedy, puede quedar en desventaja. No obstante, existirán casos en donde Mateo ganará siempre, sin importar la estrategia que utilice Sophia, tal como vimos en la sección 2.3.